Ejercicio 4: Planificacion Round Robin

Se cargan las cuatro tareas del Ejercicio 1, todas configuradas con igual prioridad y con configUSE_TIME_SLICING habilitado (Round Robin). Se capturan al menos dos hiperperiodos con el analizador logico.

Conjunto de tareas (Ejercicio 1)

Tarea	C (unid.)	T = D (unid.)	Pin
A	2	12	PB0
B	2	24	PB1
C	2	6	PB2
D	1	3	PB3

Recordatorio del Ejercicio 1: U = 2/12 + 2/24 + 2/6 + 1/3 = 22/24 = 0,917 < 1. La utilizacion es menor que 1, lo que es condicion necesaria (pero no suficiente bajo Round Robin) para la planificabilidad.

Escala de tiempos: 1 unidad = 4 ticks = 4 x 16 ms = 64 ms reales. El hiperperiodo es TP = mcm(12, 24, 6, 3) = 24 unidades = 96 ticks = 1536 ms.

a) Cuanto de tiempo (quantum) de Round Robin

En FreeRTOS con configUSE_TIME_SLICING = 1, el planificador reparte el procesador entre las tareas de igual prioridad en la cola de listos (ready list) rotandolas en cada tick del sistema. El cuanto de tiempo (quantum) de Round Robin es, por lo tanto, exactamente 1 tick del sistema.

En esta configuracion, portTICK_PERIOD_MS = 16 ms, por lo que el quantum es 16 ms (1 tick). En la captura del analizador logico se observa que, cuando varias tareas estan listas simultaneamente, cada una ejecuta durante exactamente 1 tick antes de ceder el procesador a la siguiente.

Si, el cuanto de tiempo coincide con el periodo del tick del sistema (16 ms = 1 tick).

b) Instante de finalizacion de la primera rafaga de cada tarea

En t = 0, las cuatro tareas se activan simultaneamente. Bajo Round Robin con quantum = 1 tick, el planificador las ejecuta en el orden en que fueron insertadas en la lista de listos (A, B, C, D en este caso, por orden de creacion con xTaskCreate).

Analisis tick a tick de la primera rafaga (recordando que el computo se mide en trabajo ejecutado, no en tiempo de reloj):

Cada tarea necesita C unidades x 4 ticks/unidad = C*4 ticks de CPU para completar su rafaga. Bajo RR, en cada tick solo ejecuta la tarea que tiene el turno. Con 4 tareas listas, cada una recibe 1 de cada 4 ticks.

Tarea	C (unid.)	CPU necesaria (ticks)	Ticks hasta fin (RR, 4 tareas)
A	2	8	~29
B	2	8	~30
C	2	8	~31
D	1	4	~16

Detalle del mecanismo: con 4 tareas listas en t=0, se rotan A->B->C->D->A->B->... Cada tarea recibe 1 tick de CPU cada 4 ticks de reloj. D necesita 4 ticks de CPU: los recibe en los ticks 3, 7, 11, 15 (instantes de reloj). Termina su primera rafaga alrededor del tick 16 (256 ms). A necesita 8 ticks de CPU: los recibe en los ticks 0, 4, 8, 12, 16, 20, 24, 28. Pero D libera su lugar tras el tick 15, asi que a partir del tick 16 quedan 3 tareas rotando, lo que acelera a A, B y C levemente.

Valores aproximados del instante de finalizacion de la primera rafaga (a confirmar con la captura del analizador logico):

Tarea	CPU necesaria	Fin aprox. 1ra rafaga	Fin en ms (x16)
D	4 ticks	tick ~16	~256 ms
A	8 ticks	tick ~29	~464 ms
B	8 ticks	tick ~30	~480 ms
C	8 ticks	tick ~31	~496 ms

El punto clave: las primeras rafagas tardan mucho mas que su computo C porque el procesador se reparte equitativamente entre todas las tareas listas. Una tarea con C = 2 unidades (8 ticks de CPU) no termina en 8 ticks de reloj sino en ~29-31, porque solo recibe 1 de cada 3 o 4 ticks.

c) Estadisticas del reporte por terminal

La tarea de reporte imprime periodicamente (cada 3 s) las estadisticas acumuladas. Los valores esperados para el regimen estacionario son:

Tarea	Activaciones	Plazos perdidos	Rmax (ms)	Jitter max (ms)
A	~continuas	pocos/ninguno	~480-512	~0-16
B	~continuas	pocos/ninguno	~480-512	~0-16
C	~continuas	muchos	~480-512	~0-16
D	~continuas	muchos	~256-288	~0-16

Nota: los valores exactos dependen de la simulacion en Proteus. La columna "Plazos perdidos" refleja que bajo Round Robin, las tareas C y D con periodos cortos (T=6 y T=3 unidades respectivamente) pierden plazos consistentemente. Los valores de Rmax y jitter deben leerse del Virtual Terminal de la simulacion.

d) Por que se pierden plazos si U < 1?

El factor de utilizacion U = 22/24 = 0,917 < 1 indica que el procesador tiene capacidad de computo suficiente para ejecutar todas las tareas. Sin embargo, la condicion U < 1 es necesaria pero NO suficiente para garantizar el cumplimiento de plazos. La planificabilidad depende tambien de la politica de planificacion.

Bajo Round Robin, el procesador se reparte equitativamente entre TODAS las tareas listas, sin considerar sus plazos ni sus periodos. El problema fundamental es que Round Robin ignora la urgencia de cada tarea:

- La tarea D (C=1, T=3): necesita 4 ticks de CPU para completar su rafaga, pero su plazo es de solo 12 ticks (3 unidades x 4 ticks). En t=0, con 4 tareas listas, D recibe solo 1 de cada 4 ticks. Para acumular 4 ticks de CPU necesita ~16 ticks de reloj, que excede ampliamente su plazo de 12 ticks.
- La tarea C (C=2, T=6): necesita 8 ticks de CPU, pero su plazo es de 24 ticks (6 unidades x 4). Con el reparto equitativo, necesita ~29-31 ticks de reloj, excediendo su plazo de 24 ticks.
- Las tareas A y B (T=12 y T=24): tienen plazos mas holgados (48 y 96 ticks), por lo que pueden tolerar la demora del reparto equitativo sin perder plazos (o perdiendo muy pocos).

La raiz del problema: Round Robin da el mismo tiempo de CPU a todas las tareas, independientemente de su urgencia. Una tarea con plazo corto (como D, con T=3) recibe el mismo trato que una con plazo largo (como B, con T=24). El planificador no tiene en cuenta los requisitos temporales individuales de cada tarea.

En contraste, bajo Rate Monotonic (prioridades inversas al periodo), D tendria la prioridad mas alta y ejecutaria primero, garantizando que sus plazos se cumplan. Como se demostro en el Ejercicio 1, RM planifica exitosamente este conjunto porque los periodos son armonicos y U < 1.

Conclusion: U < 1 dice que "hay suficiente CPU en promedio", pero no dice nada sobre como se distribuye esa CPU en el tiempo. Round Robin distribuye uniformemente, sin atender la urgencia, y eso provoca que las tareas mas frecuentes (con plazos cortos) no reciban CPU a tiempo.

e) Que tarea resulta mas perjudicada?

La tarea mas perjudicada por el reparto equitativo es la tarea D.

Justificacion: D es la tarea con el periodo mas corto (T = 3 unidades = 12 ticks) y por lo tanto la que tiene el plazo de respuesta mas ajustado. Su computo (C = 1 unidad = 4 ticks de CPU) es el menor de todas, pero bajo Round Robin eso no le da ninguna ventaja: recibe la misma fraccion de CPU que las demas.

Para cuantificar el perjuicio: D necesita 4 ticks de CPU y tiene un plazo de 12 ticks. En la peor situacion (4 tareas listas), recibe 1 de cada 4 ticks, necesitando 4x4-3 = 13 ticks minimos para completar su rafaga (mas la sobrecarga del OS). Esto ya supera su plazo. En la practica, D pierde plazos de forma consistente en practicamente todas sus activaciones.

Comparacion de la "holgura relativa" de cada tarea:

Tarea	C	T	C/T	Holgura relativa (T-C)/T
D	1	3	0,333	0,667 (la menor)
C	2	6	0,333	0,667
A	2	12	0,167	0,833
B	2	24	0,083	0,917 (la mayor)

D y C tienen la misma razon C/T, pero D se activa el doble de veces que C dentro de un hiperperiodo (8 vs. 4 activaciones), por lo que acumula mas plazos perdidos en terminos absolutos. Ademas, su periodo absoluto es el mas corto (3 unidades = 192 ms reales), dejandole el menor margen temporal para absorber la interferencia del reparto equitativo.

En un sistema con prioridades por tasa monotona (RM), D tendria la prioridad mas alta justamente por ser la mas frecuente. Round Robin le quita esa ventaja y la perjudica mas que a ninguna otra.

Codigo

Archivo Ejercicio4.ino. Cambios respecto del Ejercicio 3:

1. La tabla tareas[] ahora contiene las cuatro tareas (A, B, C, D) en lugar de solo D.
2. Todas las tareas se crean con la MISMA prioridad (PRIO_PERIODICAS = 2), lo que activa el mecanismo Round Robin de FreeRTOS.
3. Se requiere configUSE_PREEMPTION = 1 y configUSE_TIME_SLICING = 1 en FreeRTOSConfig.h.

El cuerpo de las tareas (vTareaPeriodica) y la tarea de reporte (vReporte) son identicos al Ejercicio 3. La estructura de datos tarea_t registra automaticamente activaciones, plazos perdidos, Rmax y jitter.

```
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <task.h>
#include <util/delay_basic.h>

#define TICKS_POR_UNIDAD   4
#define PRIO_PERIODICAS    2   /* TODAS con la MISMA prioridad */
#define PRIO_REPORTE       1
#define PERIODO_REPORTE_MS 3000

#define ITER_POR_MS  ((uint16_t)(F_CPU / 1000UL / 4UL))  /* 4000 @16MHz */

static void quemar_cpu_ms(uint16_t ms)
{
  while (ms--) _delay_loop_2(ITER_POR_MS);
}

typedef struct {
  const char *nombre;
  uint8_t     pin;
  uint8_t     C_unidades;
  uint8_t     T_unidades;
  uint32_t    activaciones;
  uint32_t    perdidos;
  uint16_t    resp_max;
  uint16_t    jitter_max;
} tarea_t;

/* Ejercicio 4: las cuatro tareas, IGUAL prioridad */
static tarea_t tareas[] = {
  { "A", PB0, 2, 12, 0, 0, 0, 0 },
  { "B", PB1, 2, 24, 0, 0, 0, 0 },
  { "C", PB2, 2,  6, 0, 0, 0, 0 },
  { "D", PB3, 1,  3, 0, 0, 0, 0 },
};
#define N_TAREAS (sizeof(tareas) / sizeof(tareas[0]))

static void vTareaPeriodica(void *pv)
{
  tarea_t *t = (tarea_t *)pv;
  const TickType_t periodo = (TickType_t)t->T_unidades * TICKS_POR_UNIDAD;
  const uint16_t computo = (uint16_t)t->C_unidades
                           * TICKS_POR_UNIDAD * portTICK_PERIOD_MS;
  TickType_t prev     = xTaskGetTickCount();
  TickType_t esperado = prev;

  for (;;) {
    const TickType_t inicio = xTaskGetTickCount();
    const uint16_t j = (uint16_t)(inicio - esperado);
```

```
    if (j > t->jitter_max) t->jitter_max = j;

    PORTB |=  (uint8_t)(1 << t->pin);
    quemar_cpu_ms(computo);
    PORTB &= (uint8_t)~(1 << t->pin);

    const uint16_t resp = (uint16_t)(xTaskGetTickCount() - esperado);
    if (resp > t->resp_max) t->resp_max = resp;
    if (resp > periodo)     t->perdidos++;
    t->activaciones++;
    esperado += periodo;
    xTaskDelayUntil(&prev, periodo);
  }
}
```

```
static void vReporte(void *pv)
{
  (void)pv;
  TickType_t prev = xTaskGetTickCount();
  for (;;) {
    xTaskDelayUntil(&prev, pdMS_TO_TICKS(PERIODO_REPORTE_MS));
    Serial.println(F("--- tarea  activ  perdidos  Rmax  jitter ---"));
    for (uint8_t i = 0; i < N_TAREAS; i++) {
      Serial.print(F("    ")); Serial.print(tareas[i].nombre);
      Serial.print(F("  "));   Serial.print(tareas[i].activaciones);
      Serial.print(F("  "));   Serial.print(tareas[i].perdidos);
      Serial.print(F("  "));
      Serial.print((uint32_t)tareas[i].resp_max * portTICK_PERIOD_MS);
      Serial.print(F("  "));
      Serial.println((uint32_t)tareas[i].jitter_max*portTICK_PERIOD_MS);
    }
  }
}

void setup()
{
  Serial.begin(9600);
  DDRB  |=  (uint8_t)(_BV(PB0)|_BV(PB1)|_BV(PB2)|_BV(PB3));
  PORTB &= (uint8_t)~(_BV(PB0)|_BV(PB1)|_BV(PB2)|_BV(PB3));
  /* Todas con la MISMA prioridad => Round Robin */
  for (uint8_t i = 0; i < N_TAREAS; i++)
    xTaskCreate(vTareaPeriodica, tareas[i].nombre, 192,
                (void *)&tareas[i], PRIO_PERIODICAS, NULL);
  xTaskCreate(vReporte, "REP", 256, NULL, PRIO_REPORTE, NULL);
}

void loop() { }
```

Decisiones de diseno y configuracion

Por que configUSE_TIME_SLICING = 1?

FreeRTOS ofrece dos modos para tareas de igual prioridad: Round Robin (configUSE_TIME_SLICING = 1) y FIFO (configUSE_TIME_SLICING = 0). En Round Robin, el planificador rota entre tareas listas de igual prioridad en cada tick. En FIFO, la tarea que toma la CPU la mantiene hasta que se bloquea o cede voluntariamente. Este ejercicio pide explicitamente RR.

Por que todas con la misma prioridad?

El ejercicio pide configurar las cuatro tareas con igual prioridad para observar el comportamiento de Round Robin. En FreeRTOS, el mecanismo de time slicing solo actua entre tareas del mismo nivel de prioridad. Si tuvieran prioridades distintas, la de mayor prioridad ejecutaria siempre primero (esquema preemptivo con prioridades fijas), sin rotacion.

Orden de creacion de las tareas

Las tareas se crean en orden A, B, C, D. En FreeRTOS, xTaskCreate inserta la nueva tarea al final de la lista de listos para su nivel de prioridad. Por lo tanto, el orden inicial de rotacion es A->B->C->D. Este orden determina que tarea ejecuta primero en t=0 y el patron de rotacion subsiguiente.

Verificacion de configUSE_PREEMPTION

configUSE_PREEMPTION = 1 es necesario. Sin el, el planificador no seria preemptivo y el time slicing no funcionaria: cada tarea mantendria la CPU indefinidamente hasta bloquearse con xTaskDelayUntil. Con preemption + time slicing, el tick ISR verifica si hay tareas del mismo nivel de prioridad esperando y, de ser asi, fuerza un cambio de contexto.

Estructura del firmware

El firmware reutiliza integramente la estructura del Ejercicio 3. Los unicos cambios son: (1) la tabla tareas[] ahora tiene 4 entradas en lugar de 1, y (2) el stack de la tarea de reporte se incremento a 256 bytes porque ahora imprime estadisticas de 4 tareas. La funcion vTareaPeriodica es generica y funciona para cualquier cantidad de tareas gracias al parametro pv que apunta a la entrada correspondiente de la tabla.